PROJECT HANDBOOK

Student C — Model Testing and Evaluation

Satellite-Based Early Warning for Glacial Lake Outburst Floods (GLOF) · Major Project — 7th Semester · 14-week (3.5-month) week-by-week guide

> **Your job in one line:** prove whether the system actually works — honestly. You design the test, you run the big Sikkim experiment, and you decide what the numbers mean. This is the part examiners ask about most, so it is the most important role on the team, not the least.

1. Before you start — set up in the first two days

- Set up **Colab**, **GitHub** and the same conda environment as A and B, so you can run their code.
- Read the three Sikkim papers listed in the resource pack (the *Science*, *Scientific Reports* and *Eos* articles) — you need to know the disaster timeline by heart.
- Download the **HMA GLOF Database** from Zenodo — this is your list of real, confirmed floods.
- Make your folder in the repository, e.g. */evaluation*, and keep every result as a saved file, never only on screen.

2. Your toolkit — what to use and how to use it properly

Tool	What you use it for	How to use it properly
HMA GLOF Database	The official record of 697 real outburst floods — your source of truth for 'did a flood actually happen'.	Filter to Indian events and check each date against a news or paper source. A wrong date makes every result wrong.
pandas + NumPy	Comparing predicted scores against real events and computing all the numbers.	Write each metric as a small function with a test, so you can trust it. Never compute a number by hand in a notebook cell you later lose.
scikit-learn (metrics)	Standard measures: precision, recall, ROC curves.	Plain accuracy is meaningless here — with one flood in years of data, always saying 'safe' would be 99.9% accurate. Use lead time and false alarms per lake per year instead.
matplotlib	All the charts in the results chapter.	Every chart needs axis labels and units. Save charts as files from code, not screenshots — you will regenerate them many times.
pytest	Small tests that check your own evaluation code is correct.	Write one test that deliberately leaks future data and confirms your code catches it. This is your protection against the worst possible mistake.
Jupyter / Colab	Running experiments and keeping a record of each one.	Number your experiment notebooks (exp01, exp02…) and never edit an old one after reporting its result.
Git	Version control for your evaluation code and results.	Commit result files together with the code that made them, so any number can be traced back.

3. Your week-by-week plan

Fourteen weeks — three and a half months. Each row is one week. If a week slips, tell the team in the stand-up the same week — do not quietly carry it forward.

Month 1 — Deciding how you will judge the system

Week	Focus	What to do	Done when…
Week 1	**Learn the disaster**	• Read the Sikkim papers and write a one-page timeline of what happened on 4 October 2023. • Note what was visible beforehand and roughly when. • List the questions the examiner is likely to ask about it. **Tools:** Papers from the resource pack, notes	Sikkim timeline written
Week 2	**Write the testing rules**	• Write down the rules for a fair test before anyone sees results — this document protects the whole project. • The key rule: when testing whether the system would have warned about a flood, only data published **before** that date may be used. • Decide the measures: how early a warning came (lead time), how many false alarms per lake per year, and how often the system says 'cannot assess'. **Tools:** Word/Markdown, team discussion	Testing protocol document
Week 3	**Build the truth list**	• Pull all Indian events out of the HMA GLOF database. • Check each date against a second source. • Mark which of our study lakes have a confirmed event and which are calm control lakes. **Tools:** HMA GLOF Database (Zenodo), pandas	Verified list of real events
Week 4	**Decide the splits**	• Decide how the model will be tested on lakes it has never seen (leave one lake out). • Fix the date cut-offs for training and testing, and get the whole team to agree in writing. **Tools:** pandas, protocol document	Test plan signed off by the team

Month 2 — Building the testing machine

Week	Focus	What to do	Done when…
Week 5	**Write the harness**	• Write the code that takes any set of scores and produces all your measures and charts. • Test it first on fake made-up scores so you know the code itself is correct. **Tools:** pandas, scikit-learn, matplotlib, pytest	Evaluation harness working on fake data
Week 6	**First real numbers**	• Run the harness on B's simple baseline scores. • Produce the first results table — even if the numbers are poor, this is a milestone. **Tools:** Harness, B's baseline scores	First baseline results table
Week 7	**Guard against cheating**	• Write a test that checks no future data was used. • Deliberately feed in leaked data and confirm your check catches it. • Review the whole chain with A to be sure the dates are clean. **Tools:** pytest, pandas	Leak check in place and proven
Week 8	**Baseline report**	• Write a short internal report: what works, what does not, which signals look useless. • Present it at the mid-project review — this steers B's next month of work. **Tools:** matplotlib, Word/Markdown	Baseline report presented

Month 3 — The big experiment

Week	Focus	What to do	Done when…
Week 9	**Test the real models**	• Run B's anomaly models through the harness. • Produce the lead-time chart: if the system had flagged the lake, how many weeks in advance? **Tools:** Harness, B's model scores	Lead-time results
Week 10	**The Sikkim test**	• Run the headline experiment on **South Lhonak Lake** using only data available before 4 October 2023. • Report the result honestly, whether it is positive or negative. • Write down exactly which data files were used, so it can be repeated. **Tools:** A's pre-event data slice, harness	Sikkim retrospective result — the headline
Week 11	**Test on unseen lakes**	• Train on some lakes and test on lakes the model has never seen. • This shows whether the system has genuinely learned something or just memorised. **Tools:** Harness, scikit-learn	Generalisation results
Week 12	**Failures, and which signals matter**	• Look closely at the false alarms — what caused each one? Write up two or three as short stories with charts; examiners love concrete examples. • With B, run the model repeatedly with one signal removed each time and record how much each one contributes. • With A, measure how many extra observation days radar gives during the cloudy monsoon — a headline number of the project. **Tools:** matplotlib, pandas, MLflow	Failure cases, ablation and radar-coverage results

Final two weeks — final numbers and writing

Week	Focus	What to do	Done when…
Week 13	**Set the final thresholds**	• Decide the cut-off at which the system says 'cannot assess' rather than giving a score. • Produce the final, locked results tables. No changing numbers after this week. **Tools:** pandas, harness	Final results locked
Week 14	**Charts, chapter and final check**	• Produce every final figure for the report at proper quality. • Write the 'Results and Discussion' chapter in simple language — say what the numbers mean, not just what they are. • Re-run every reported number from scratch and confirm it matches the report. • Prepare to answer: 'how do you know the system did not secretly see the future?' **Tools:** matplotlib, Word/LaTeX, Git, checklist	Chapter written; every number verified

4. Who you depend on, and who depends on you

Person	What passes between you
Student A (Data Collection)	A gives you the strictly pre-October-2023 data slice and the radar coverage counts. You check the dates are truly clean.
Student B (Model Development)	B gives you scores in the agreed format. You give B failure analysis so the model can improve.
Student D (App Development)	You give D the final figures and numbers for the report and the paper.

5. Common mistakes to avoid

- Using accuracy as your measure — with one flood in years of data, always saying 'safe' would score 99.9%.
- Accidentally letting the model see data from after the flood date; this invalidates the whole result and reviewers check for it.
- Reporting only the good runs. Negative results are still publishable here — hiding them is not.
- Leaving your numbers in a notebook that gets overwritten, so you cannot reproduce them later.
- Starting the Sikkim test late. It is the centre of the viva — it must be finished by the end of month 3.

6. What you write in the final report

- Results — all measures, tables and charts.
- Analysis — what the numbers mean, including the lead-time curve and false-alarm rate.
- Discussion — why the system succeeds or fails, with concrete case studies.
- The evaluation protocol, showing the test was fair and repeatable.

Every week ends with the team stand-up: say what you finished, what is stuck, and what is next. Push your work to the shared GitHub repository the same day — never let the only copy sit on your laptop.